

STANDALONE RESEARCH ON MIROSHARK – MANAGING HUNDREDS OF AGENTS

Repo Overview

MiroShark is a **Universal Swarm Intelligence Engine**. You feed it a scenario (document, news headline, policy draft, historical what-if, etc.), and it spawns hundreds of grounded agents that simulate reactions hour by hour across Twitter, Reddit, prediction markets, etc. It is designed for cheap, fast, large-scale multi-agent simulations (\$1 and <10 min for basic runs).

Core Architecture for Scaling Hundreds of Agents

- **Graph Memory Layer:** Neo4j (local or Aura) for agent relationships, interaction networks, echo chambers, and archetype clustering.
- **Agent Model:** Hundreds of personas with archetypes (analyst, influencer, retail, observer, etc.). Each agent has persistent memory and can invoke MCP tools.
- **Simulation Engine:** Hour-by-hour stepping with live data injection (RSS, breaking news). Supports counterfactual branching and Director Mode.
- **Backend:** Python + FastAPI, with Docker + docker-compose for easy scaling.
- **Frontend:** React/Next.js dashboard for scenario input, live monitoring, and results (interaction graph, demographics, quality diagnostics).
- **Orchestration:** Background job queuing for long-running simulations, with push notifications on completion.
- **Cost Control:** Cheap vs Best presets (model routing via OpenRouter or local Ollama).

Key Insights for Managing Hundreds of Agents

1. Graph-Centric Memory & Relationships

MiroShark uses Neo4j to model agent-to-agent interactions as a force-directed graph. This allows efficient querying of echo chambers, influence propagation, and demographic clustering.

Insight for Sovereign Machina: Integrate a similar graph layer into GBrain / MemPalace (using sqlite-vec or a lightweight Neo4j instance on the Hermes Pod). This would deepen relational memory for voice notes, screen awareness, and RKG entities.

2. **Archetype-Based Persona Scaling**

Agents are not generic — they are clustered into archetypes with distinct behaviors. This makes large swarms manageable and realistic.

Insight for Sovereign Machina: Extend Hermes skill distillation to generate archetype-based sub-agents. This would allow scalable background tasks (e.g., “simulate 50 personas reacting to this research note”) without exploding resource usage.

3. **Background Job Queuing & Observability**

Long-running simulations run as background jobs with real-time observability (event stream, quality diagnostics, trace interview).

Insight for Sovereign Machina: Adopt a similar Minion-style queue in the Hermes Pod for durable, resumable tasks. Combine with your existing Ouroboros auditing for full traceability.

4. **Live Data Injection & Branching**

Ability to inject breaking news mid-simulation or fork timelines.

Insight for Sovereign Machina: Apply this to voice-driven workflows in Machina Terminal — e.g., speak “what if X happened?” and trigger a branched simulation in the background.

5. **Cost & Scalability Controls**

Preset-based model routing and cheap/local options keep costs low even with hundreds of agents.

Insight for Sovereign Machina: Use similar presets in your GEPA/ Hermes configuration to balance quality vs resource usage on the K15 Hermes Pod.

Relevance to Sovereign Machina

MiroShark shows that managing hundreds of agents is feasible with graph memory, archetype personas, background queuing, and observability. These patterns can deepen your Hermes supervisor and GBrain rewrite without breaking your tri/quad-node invariants. The graph-centric approach is especially useful for relational knowledge in MemPalace and RKG.

ARCHITECT SIGN-OFF

MiroShark provides strong, practical insights for scaling agent swarms: graph-based

memory, archetype personas, background job queuing, live injection, and observability. These can be adapted into your Hermes Pod and GBrain rewrite to enhance persistent multi-agent capabilities while keeping everything local-first and sovereign. The repo validates that structured memory and orchestration are the keys to managing large numbers of agents efficiently.

Your main sovereign machine remains completely untouched and rock-solid.

Clarifying questions (mandatory before next phase):

1. Do you want a concrete design for integrating graph-based memory (Neo4j-inspired) into GBrain on the Hermes Pod?
2. Shall I outline how to add archetype-based persona scaling to Hermes for background simulations?

Reply with your priority and I will deliver the focused design. The agent scaling insights are ready to be applied.